

Kactus2 aids in the creation of hardware designs with hierarchy, software architecture in MCAPI communication abstraction and system designs that map software to hardware. It configures all designs, i.e., it sets the requirements, constraints, optimisation, active views, etc. This tool generates structural top-level VHDL and performs software builds for all processors. It also creates combined documentation from all blocks, through all hierarchy levels.

However, Kactus2 cannot create IP functionality. HDL editors and integrated development environment (IDE) software should be used for that. TCE tools may also be used to create C-to-VHDL so that IP can be created from C source without knowing any HDL. Another drawback is that it doesn't allow the generation of binary and executable files either. Additional HDL synthesis and software compilation tools should be used for this purpose.

System requirements

Operating system: Linux, Windows 7, Windows Vista or Windows XP.

Installation

The installation is simple and takes only minutes. Download the *.exe file that your OS supports. Double clicking on it opens the installation window. Go through the procedure to install Kactus2 on your computer.

Configure library paths

Clicking on the Configure Library icon from the Library in the top pane opens a dialogue box. Browse and add the location where you downloaded the library from opencores.org to make sure that you can access the files created from different computers. This should be set every time Kactus2 is accessed from a different computer.

Create a bus

The bus is pre-given in the exercise library.

To create a new bus, go to File, select New, and in the new window that



Fig. 3: Creating a new HW component

appears, select Bus from the left hand menu. On the right side will be several VLVN fields, as shown in Fig. 2.

According to the IP-XACT format, each object should be given a unique identifier defined by the fields' vendor, library, name and version. In this example, we are using the naming convention specified in Set Vendor to 'Altera.' Use 'ip.hwp.communication' for the Library field, 'avalon_bus' for the Name field, and '1.0' for the Version field. Click OK to create the new bus.

Normally, after creating the bus, we define the bus signals in the appeared 'avalon (1.0) [bus]' tab. But this example focuses on creating hierarchical system and documentation features of Kactus2. So now we can create buses without any signals; hence, close that tab.

Creating a hardware component

This is similar to creating a new bus. Go to File→New. Select HW Component

from the left hand menu of the pop-up window. The Product Hierarchy option lets you choose which type of component you are specifying; select IP from the pull down menu. The Firmness option gives information on how the component can be modified; select Template from this pull-down menu.

Fill in the VLVN fields with your G<group number> (group 3 would use G03, etc...), 'ip.hwp.cpu,' 'nios2,' and '1.0' as given in Fig. 3. Create the 'hardware component' by pressing OK.

After creating the component, a new tab opens which is filled with information about the component. In the given example, the only bits of information to be filled for each component are the bus interfaces and CPU details.

Adding interfaces to components

We only need 'avalon_bus' for the Nios2 processor component. From the Nios2 tab, select Bus Interfaces. Double click on the empty list space to create a new bus interface and type 'Avalon_Master' in the name field. You can also give a display name and description to this interface.

It is to be noted that you are not able to edit the Bus Definition column in the Bus Interfaces list. Open the bus interface by going to the tree view: Bus Interfaces→Avalon Master and double click on it. In order to tie the bus interface to the actual bus, we should fill in the Bus Definition and Abstraction Definition fields. This is done by using the drag and drop method from the hierarchy pane. Expand 'avalon_bus' from the hierarchy pane to show the Abstraction

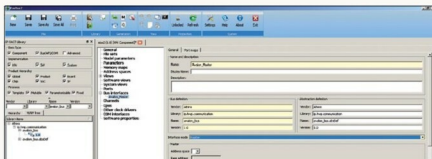


Fig. 4: Adding interfaces to components